Submission Worksheet

IT202-450-M2026 / it202-milestone-1-2026

Submission Data

Course	IT202-450-M2026
Assignment	IT202 Milestone 1 - 2026
Student	Shakir A. (sa3327)
Status	submitted
Worksheet Progress	100%
Potential Grade	NaN/NaN (NaN%)
Updated	7/13/2026, 8:29:42 PM

Grading Link
https://courses.ethereallab.app/assignment/v3/IT202-450-M2026/it202-milestone-1-2026/grading/sa3327

View Link
https://courses.ethereallab.app/assignment/v3/IT202-450-M2026/it202-milestone-1-2026/sa3327

Instructions

Verify that your lesson-built Milestone 1 account system is complete and functional, then collect focused evidence for the account, access, role, profile, and styling requirements. Most of the coding should already be done from the lessons; this worksheet focuses on confirming the required behavior, polishing the project CSS, and explaining your understanding.

Assignment Setup

- Required branch: `Milestone1`
- Required starting branch: `qa`
- Required project folder: `public_html/project/`
- Required deployed evidence target: working `qa` site before production promotion
- Recommended order:
  - Read the whole worksheet.
  - Review the Milestone 1 assignment requirements.
  - Check out `qa`, pull the latest version, and create the milestone branch.
  - Ensure the related lessons are complete in your project.
  - Test each feature and requirement.
  - Apply your custom CSS to replace the temporary lesson example CSS.
  - Merge the milestone branch into `qa`, test deployed behavior, then collect worksheet evidence.

Git Workflow Checklist

1. Check out `qa`.
2. Pull the latest `qa` branch with `git pull origin qa`.
3. Create the milestone branch with `git checkout -b Milestone1`.
4. Commit focused verification, fixes, and styling updates in small pieces. If there's nothing to commit, wait until you add the PDF to your repository.
5. Push `Milestone1`.
6. Open a pull request from `Milestone1` into `qa` and keep it open while you work.

you add the PDF to your repository.

5. Push `Milestone1`.
6. Open a pull request from `Milestone1` into `qa` and keep it open while you work.
7. After all Milestone 1 requirements pass locally, merge the pull request into `qa`.
8. Gather evidence from the deployed `qa` site and fill out the worksheet.
9. Export the worksheet PDF from the Learn Courses Platform.
10. Save the PDF in the repository under `docs/milestone01/`.
11. Add, commit, and push the PDF to the `Milestone1` branch.
    - `git add .`
    - `git commit -m "Add milestone 1 worksheet PDF"`
    - `git push origin Milestone1`
12. Upload the same PDF to Canvas.
13. Create and merge a pull request from `qa` into `prod` when production evidence is required.
14. Switch back to `qa`.
15. Pull the latest `qa` branch.

Shared Requirements

Apply these requirements to every Milestone 1 feature:

- Add a comment with your UCID, date, and a brief summary of the code block or feature area.
- Follow the shared helper, validation, flash message, session, and role-check patterns from the lessons.
- Replace the temporary lesson CSS with your own project styling.
- Use user-friendly flash messages for success and failure states.
- Log technical errors with `error_log()` instead of showing raw technical output to users.
- Use HTML validation attributes, JavaScript validation through the page's `validate()` function, and PHP validation before database writes.
- Keep safe form values sticky after validation errors.
- Clear password fields after validation errors.
- Test pages through the supported deployed `qa` URLs before collecting browser evidence.
- Use your own code, database behavior, branch, pull request, deployed URLs, and explanation evidence.

Evidence Format

Most sections use the same evidence pattern:

- Images:
  - Show relevant code and matching browser output.
  - Make sure UCID/date comments are visible in code evidence.
  - Make sure the deployed `qa` URL is visible in browser output evidence.
  - One combined screenshot is acceptable when the code and output are both readable.
  - Use multiple screenshots when one image would be hard to read.
- URLs:
  - Provide direct GitHub file links from the milestone branch.
  - Provide anticipated production URLs for deployed PHP pages.
  - Capture the tested `qa` URL, change `-qa` to `-prod`, and keep the same path and filename.
  - PHP page URLs must end in `.php`; zero credit for that URL entry if it does not.
- Response:
  - Explain the logic in your own words.
  - Describe the steps or decisions your PHP, SQL, or JavaScript makes.
  - Do not restate the prompt or list the requirements.

Submission Check

- Confirm the Milestone 1 branch has been pushed.
- Confirm the pull request into `qa` exists.

Submission Check

- Confirm the Milestone 1 branch has been pushed.
- Confirm the pull request into `qa` exists.
- Confirm meaningful Milestone 1 commits are visible.
- Confirm the deployed `qa` version was reviewed before promotion.
- Confirm registration, login, logout, protected access, role access, profile edit, and password change were tested.
- Confirm the temporary lesson CSS has been replaced with your own styling.
- Confirm anticipated production URLs use the same path as the tested `qa` URLs with `-qa` changed to `-prod`.
- Confirm the worksheet PDF is committed under `docs/milestone01/` on the milestone branch.
- Upload the same worksheet PDF to Canvas.

Submission Progress 100%

100%

`Users`

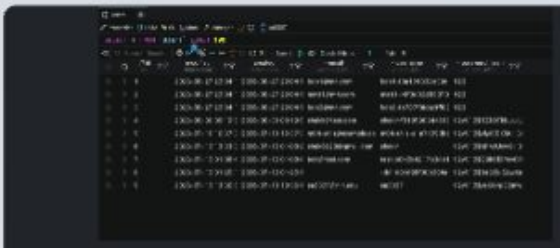
`Roles`
`UserRoles`

100%

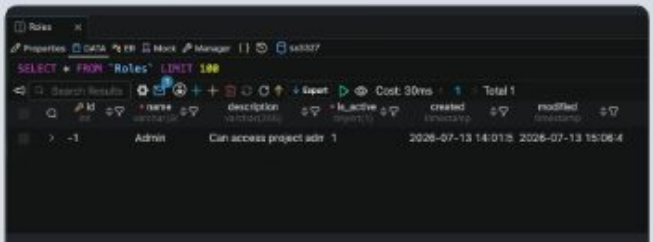
100%

ⓘ Details

- Show the `Users` table from the VS Code database extension so the structure and some data are visible.
- Show the `Roles` table from the VS Code database extension so the structure and some data are visible.
- Show the `UserRoles` table from the VS Code database extension so the structure and some data are visible.



Users



Roles

UserRoles

100%

100%

ⓘ Details

- Paste the direct GitHub file link for `sql/001_create_users.sql`.
- Paste the direct GitHub file link for `sql/003_create_table_roles.sql`.
- Paste the direct GitHub file link for `sql/004_create_table_userroles.sql`.

https://github.com/shakirahmed3317/sa3327-it202-450-m2026/blob/Milestone1/sql/001_create_users.sql

https://github.com/shakirahmed3317/sa3327-it202-450-m2026/blob/Milestone1/sql/003_create_table_roles.sql

https://github.com/shakirahmed3317/sa3327-it202-450-m2026/blob/Milestone1/sql/004_create_table_userroles.sql

100%

ⓘ Details

- Explain how the three tables work together.
- Explain where username and email uniqueness are enforced.

Section #2: (1 pt.) Shared Setup And Helpers

100%

- Goal: show that the project loads and uses reusable account helpers from a shared setup file.
- Expected helper behavior:
  - Shared app setup loads the database, session, validation helpers, user helpers, role helpers, flash helpers, and other project utilities.
  - User helper functions safely check login state before reading user session values.
  - Role helper functions can check the current user's roles.

Task #1 (1 pt.) – Shared Helper Evidence

100%

Combo #1

Part 1: Images 100%

ⓘ Details

- Show `lib/app.php` with the shared helper imports visible.
- The screenshot should show references to the database, session, validation, flash, user, role, render, duplicate-user-detail, and URL helper files.

app.php

Part 2: URLs 100%

ⓘ Details

- Paste the direct GitHub file link for `lib/app.php`.

URL #1

https://github.com/shakirahmed3317/sa3327-it202-450-m2026/blob/Milestone1/lib/app.php

Part 3: Response 100%

ⓘ Details

- Explain how `lib/app.php` loads the shared helper files.
- Explain why the specific load order matters.

Response

All of the shared helper files have been included by requireonce in the lib/app.php. By using requireonce it avoids files being included more than once and they can be used across the application. Load order is important because other helper files depend on functions from earlier files. Rolehelpers.php for instance relies on functions in userhelpers.php, flashmessages.php and urlhelpers.php.

Supports **bold**, *italic*, [links](url), `code`, etc. 394 chars

Section #3: (2 pts.) Registration

100%

- Goal: show that users can register with username, email, password, and confirm password.
- Expected behavior:
  - HTML, JavaScript, and PHP validation are all present.
  - Duplicate username and duplicate email cases show specific, user-friendly feedback.
  - Passwords are stored as hashes.
  - Safe form values remain sticky after validation errors.

Task #1 (2 pts.) – Registration Evidence

100%

Combo #1

Part 1: Images 100%

ⓘ Details

- Show the registration form code, including username, email, password, confirm password, and the escaped username pattern.
- Show the JavaScript `validate()` logic for registration.
- Show the PHP validation and insert logic, including password hashing and duplicate handling.
- Show deployed `qa` browser evidence for successful registration, duplicate username, duplicate email, and one validation failure.

- Show the PHP validation and insert logic, including password hashing and duplicate handling.
- Show deployed qa browser evidence for successful registration, duplicate username, duplicate email, and one validation failure.

form code

```
<script>
    function validate(form) {
        const errors = [];

        validate_email(form.email, errors);
        validate_username(form.username, errors);
        validate_password(form.password, errors);
        validate_passwords_match(form.password, form.confirm_password, errors);

        return show_validation_errors(errors);
    }
</script>
```

validate

php validation

registration successful

duplicate email

duplicate user

validation err

Part 2: URLs 100%

ⓘ Details

- Paste the direct GitHub file link for `public_html/project/register.php`.
- Paste the anticipated production URL for the registration page.

URL #1
https://github.com/shakirahmed3317/sa3327-it202-450-m2026/blob/Milestone1/public_html/project/register.php 👍

URL #2
https://sa3327-it202-450-m2026-prod.onrender.com/project/register.php 👍

Part 3: Response 100%

ⓘ Details

- Explain how registration validates input across HTML, JavaScript, and PHP.
- Explain how duplicate username and email feedback is shown.
- Explain how the password is stored safely.

- Explain how duplicate username and email feedback is shown.
- Explain how the password is stored safely.

Response

HTML uses attributes like required, pattern, minlength for basic validation.

JS checks input for the form.

PHP does the same validation on the server to make sure that invalid data cannot bypass the client-side checks.

If there is a duplicate username or email, the database throws a error and a flash messagae is displayed to the user.

THe password is safely stored by hashing it and using the PASSWORD_BYCRUPY algorithm before its saved to the DB.

Supports **bold**, *italic*, [links](url), `code`, etc. 454 chars

Section #4: (2 pts.) Login And Logout

100%

- Goal: show that users can log in by username or email and fully log out.
- Expected login behavior:
  - Login uses one identifier field.
  - The server-side lookup safely checks username or email.
  - Passwords are verified with `password_verify()`.
  - The session stores the user's safe details and roles.
- Expected logout behavior:
  - Logout destroys the session.
  - Logout redirects with a friendly flash message.
  - Reloading a protected page after logout does not allow access.

Task #1 (2 pts.) – Login And Logout Evidence

100%

Combo #1

Part 1: Images 100%

Details

- Show the login form code with the single username-or-email identifier field.
- Show the JavaScript `validate()` logic for login.
- Show the PHP login lookup, password verification, and session assignment logic.
- Show deployed `qa` browser evidence for login with email, login with username, invalid credentials, logout, and protected access after logout.
- For the login evidence, capturing the network request in developer tools is a good way to show the differences

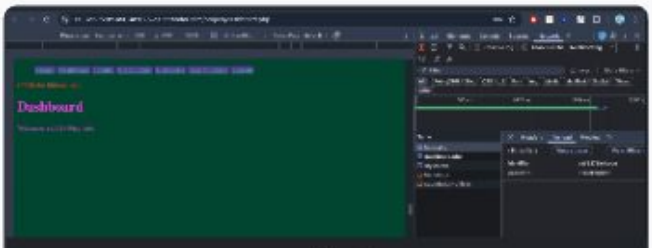
login form

validate

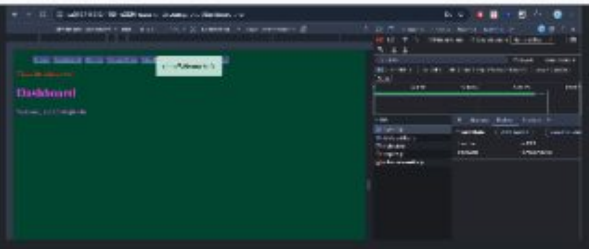
login form

validate

php validation



email login



username login

invalid creds

logout network tab (not sure really how make a picture showing the logout stuff)

Part 2: URLs 100%

ⓘ Details

- Paste the direct GitHub file link for `public_html/project/login.php`.
- Paste the direct GitHub file link for `public_html/project/logout.php`.
- Paste anticipated production URLs for the login and logout pages.

URL #1
https://github.com/shakirahmed3317/sa3327-it202-450-m2026/blob/Milestone1/public_html/project/login.php

URL #2
https://github.com/shakirahmed3317/sa3327-it202-450-m2026/blob/Milestone1/public_html/project/logout.php

URL #3
https://sa3327-it202-450-m2026-prod.onrender.com/project/login.php

URL #4
https://sa3327-it202-450-m2026-prod.onrender.com/project/logout.php

Part 3: Response 100%

ⓘ Details

- Explain how the identifier can match either username or email.
- Explain how the password check works.
- Explain how logout clears access.

Response

The login check validates input to the username and the email fields, thus user will have access via email or username. Passwordverify() compares input password with password in database and login will

The login check validates input to the username and the email fields, thus user will have access via email or username. Passwordverify() compares input password with password in database and login will succeed if it match. Logout with sessionunset() and session_destroy() and redirect user to login form page.

Supports **bold**, *italic*, [links](url), `code`, etc. 309 chars

Section #5: (2 pts.) Profile

100%

- Goal: show that users can view and update profile information safely.
- Expected account-details behavior:
  - Profile view shows safe account information.
  - Username and email can be updated without the current password.
  - Duplicate username and email cases show specific, user-friendly feedback.
  - HTML, JavaScript, and PHP validation are all present.
- Expected password-change behavior:
  - Current password is required before saving a new password.
  - New password and confirm password are validated.
  - Successful password changes store a new hash.

Task #1 (2 pts.) – Profile Evidence

100%

Combo #1

Part 1: Images 100%

ⓘ Details

- Show the profile view and edit form code.
- Show the account-details update logic.
- Show the password-change logic.
- Show deployed qa browser evidence for profile view, account-details update, duplicate profile feedback, password-change failure with wrong current password, and successful password change.

form code

acc update code

password change logic

profile view

duplicate profile

update success

wrong curr pass

password change success

Part 2: URLs 100%

ⓘ Details

- Paste the direct GitHub file link for `public_html/project/profile.php`.
- Paste the anticipated production URL for the profile page.

URL #1

https://github.com/shakirahmed3317/sa3327-it202-450-m2026/blob/Milestone1/public_html/project/profile.php

URL #2

https://sa3327-it202-450-m2026-prod.onrender.com/project/profile.php

Part 3: Response 100%

ⓘ Details

- Explain the flow/process for changing username/email.
- Explain why password changes do require the current password.
- Explain how updated profile values persist after refresh.

Response

If email or username is updated, the input is validated then the User table updates, the new values will also be saved in the session.

Password change requires the current password to confirm ownership of the account and it uses password_verify() to match the hash.

The updated profile values persist because they are saved in the DB and loaded again.

Supports **bold**, *italic*, [links](url), `code`, etc. 354 chars

Section #6: (1 pt.) Access Roles And Navigation

100%

- Goal: show that access rules are enforced by the server, reflected in navigation, and presented with your own project styling.
- Expected behavior:
  - Logged-out users cannot access protected pages.
  - Users without the required role cannot access role-protected pages.
  - Navigation changes based on login state.

- Users without the required role cannot access role-protected pages.
- Navigation changes based on login state.
- The project uses customized CSS instead of the temporary lesson example styling.
- Role pages from the lessons, such as create/list/assign roles, can satisfy the role-protected workflow.

Task #1 (0.67 pts.) – Access And Role Evidence

100%

Combo #1

Part 1: Images 100%

ⓘ Details

- Show the protected-page guard code.
- Show the role-protected guard code.
- Show the navigation code that changes based on login state.
- Show deployed `qa` browser evidence for a logged-out protected-page block, a wrong-role block, and a navigation state change.

role protected

admin nav

logged out

wrong role

Part 2: URLs 100%

ⓘ Details

- Paste the direct GitHub file link for `public_html/project/dashboard.php`.
- Paste the direct GitHub file link for `partials/nav.php`.
- Paste the direct GitHub file link for one role-protected admin page you tested.
- Paste anticipated production URLs for the protected and role-protected pages.

URL #1
https://github.com/shakirahmed3317/sa3327-it202-450-m2026/blob/Milestone1/public_html/project/dashboard.php

URL #2
https://github.com/shakirahmed3317/sa3327-it202-450-m2026/blob/Milestone1/partials/nav.php

URL #3
https://github.com/shakirahmed3317/sa3327-it202-450-m2026/blob/Milestone1/public_html/project/admin/assign_roles.php

URL #4
https://sa3327-it202-450-m2026-prod.onrender.com/project/admin/assign_roles.php

URL #5
https://sa3327-it202-450-m2026-prod.onrender.com/project/admin/create_role.php

URL #6

URL #5
https://sa3327-it202-450-m2026-prod.onrender.com/project/admin/create_role.php

URL #6
https://sa3327-it202-450-m2026-prod.onrender.com/project/admin/list_roles.php

Part 3: Response 100%

Details

- Explain how the protected page checks login state.
- Explain how the role-protected page checks role access.
- Explain how navigation decides what to show.

Response

The protected dashboard uses isloggedin() to see if the user has logged in, and will redirect them to the login page if they have not. The admin page uses requirerole("Admin") so only users with the role "Admin" will have access to it. The navigation menu uses isloggedin() to display login or account links, and hasrole("Admin") to display the links to the admin pages to users with the "Admin" role only.

Supports **bold**, *italic*, [links](url), `code`, etc. 406 chars

Task #2 (0.33 pts.) – Project Styling Evidence

100%

Combo #1

Part 1: Images 100%

Details

- Show the shared stylesheet or custom CSS changes.
- Show deployed `qa` browser evidence that the project no longer uses the temporary lesson example styling.

new css

qa new css

Part 2: URLs 100%

Details

- Paste the direct GitHub file link for `public_html/project/styles.css`.
- Paste the anticipated production URL for one styled project page.

URL #1
https://github.com/shakirahmed3317/sa3327-it202-450-m2026/blob/Milestone1/public_html/project/styles.css

URL #2
https://sa3327-it202-450-m2026-qa.onrender.com/project/profile.php

Part 3: Response 100%

Details

- Explain what you changed in the CSS to make the project your own.

Response

I changed the colors to make it modern and changed the font to Arial. I also removed unnecessasry animations.

Supports **bold**, *italic*, [links](url), `code`, etc. 109 chars

Section #7: (1 pt.) Misc

67%

Task #1 (0.33 pts.) – GitHub Details

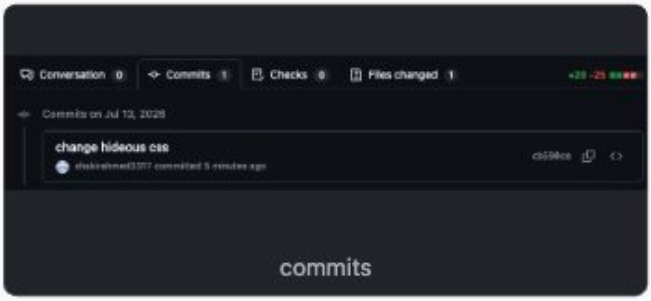
100%

Combo #1

Part 1: Images 100%

Details

- Screenshot the Commits tab of the pull request.
- The screenshot should show meaningful feature commits for Milestone 1.



commits

Part 2: URLs 100%

Details

- Paste the pull request URL.
  - The URL must end in `/pull/#`, where `#` is the pull request number; zero credit for this URL if it does not.

URL #1

https://github.com/shakirahmed3317/sa3327-it202-450-m2026/pull/51

Task #2 (0.33 pts.) – WakaTime Activity

100%

Details

ⓘ Details

- Visit the WakaTime dashboard.
- Open `Projects`.
- Find your repository.
- Screenshot the overall time near the repository name.
- Screenshot the file-level activity.
- The purpose is evidence of activity; the duration is not tied to the grade.
- The visual graphs are not required.

waka time

waka files

☰ Task #3 (0.33 pts.) – Reflection

ⓘ Details

- Answer each question with at least a few reasonable sentences.

Task #1 (0.11 pts.) – What did you learn during this assignment?

100%

Response

I learned about client side & server side validation and to store secure data, session management, and to hash password for secure and to grant user roles accordingly.

Supports **bold**, *italic*, [links](url), `code`, etc. 167 chars

Task #2 (0.11 pts.) – What was the easiest part of the assignment?

100%

Response

The easiest part was understanding the HTML side of things, it was just basic forms and elements.

Supports **bold**, *italic*, [links](url), `code`, etc. 98 chars

⧗ Saving...

Task #3 (0.11 pts.) – What was the hardest part of the assignment?

100%

Response

The hardest part was grasiping the way the validation happened between HTML, JS, and PHP and making

Response

The hardest part was grasiping the way the validation happened between HTML, JS, and PHP and making sure all three worked together. It also took some time to understand how the helper functions were doing and how they displayed the errors.

Supports **bold**, *italic*, [links](url), `code`, etc. 239 chars

End of Assignment - submission has been recorded.